MCO-1D ORNITHOPTER

Commissioning Manual — Revision 15

The complete guide to the MCO-1D from first assembly through experimental discovery flights. Written assuming you know nothing about this project. Every term is defined when first used. Every step is numbered and explained. Do not skip steps.

Aircraft	MCO-1D metachronal cascade ornithopter
Mission	Search and Rescue testbed, biomimetic flight research
MTOW	3.448 kg (7.60 lb)
Stall speed	4.85 m/s (9.4 kt) out of ground effect
VNE	28.0 m/s (54.4 kt)
Wings	8 independent — four tandem pairs W1 (front) to W4 (rear)
Motors	8 × GoolRC 5010-14 360 KV — one per wing
ODrives	4 × v3.6 — axis0 = port, axis1 = starboard per pair
Per-wing sensing	AoA (MCP3208 #1), Sweep (MCP3208 #2), Stroke (ODrive CAN)
Wave Library	45,229 lines Python — 29 hypotheses, 114 GS commands
Tick rate	50 Hz (20 ms per tick)
Pi 4 tick time	~13 ms mean — 1.5× headroom
Revision	15 — July 2026

TABLE OF CONTENTS

PART I — WHAT THIS IS AND HOW IT WORKS

1. What the MCO-1D Is
2. How the Eight Wings Work Together
3. The Sensing System — What the Computer Knows
4. How the Software Controls the Aircraft
5. Safety Rules

PART II — HARDWARE — EVERY COMPONENT EXPLAINED

6. Wing Assemblies — Motor, Rope, Sector Wheel, Wing
7. ODrive Motor Controllers — 4 Boards, 8 Axes
8. Pitch Servos and Angle of Attack
9. Sweep Servos and Fore-Aft Position
10. The ADC Chips — Per-Wing Feedback Sensing
11. Flight Computer — Raspberry Pi 4
12. Sensors — IMU, Barometer, Rangefinder, OAT
13. Power System
14. Radio System — RC and Pixhawk
15. Panel and Ground Station
16. Wiring — CAN Bus, SPI Bus, GPIO

PART III — ASSEMBLY SEQUENCE

17. Assembly Steps 1 Through 24

PART IV — SOFTWARE INSTALLATION

18. Step 1 — Install Raspberry Pi OS
19. Step 2 — Configure /boot/config.txt
20. Step 3 — Install Python Packages
21. Step 4 — Copy Software Files
22. Step 5 — Configure the CAN Bus
23. Step 6 — Configure ODrives
24. Step 7 — Verify Everything Works
25. Step 8 — Set Up the Ground Station Laptop

PART V — COMMISSIONING TESTS

26. TC-01 — Bench Test: All 16 Steps
27. TC-02 — First Hover: All 8 Steps
28. TC-03 — Forward Flight: All 6 Steps

PART VI — FLIGHT OPERATIONS

29. Pre-Flight Checklist
30. Normal Flight Procedures
31. Emergency Procedures
32. Post-Flight Procedure

PART VII — DISCOVERY PROGRAMME

33. What Discovery Mode Does
34. Running a Discovery Flight
35. The 29 Hypotheses

36. Water Operations

PART VIII — REFERENCE

37. GPIO and SPI Pin Reference
38. ODrive Axis Assignment
39. Ground Station Commands
40. Software File Reference
41. Calibration Items Pending TC-02

PART I — WHAT THIS IS AND HOW IT WORKS

SECTION 1 — WHAT THE MCO-1D IS

The MCO-1D is a prototype aircraft that flies by flapping its wings, the same way a bird or insect does. It is not a conventional plane (which uses fixed wings and propellers) and it is not a multirotor drone (which uses spinning rotors). It flaps.

It has eight wings arranged in four pairs from front to rear. The front pair is called W1. The second pair is W2. Then W3 and W4 at the back. Each pair has a port (left) wing and a starboard (right) wing. So: W1 port, W1 starboard, W2 port, W2 starboard, and so on to W4 starboard — eight wings total.

Every wing is completely independent. Each has its own motor. Each motor is separately commanded by the computer fifty times per second. This is what makes the MCO-1D different from any previous ornithopter: each of the eight wings can do something different at the same moment.

The purpose of this aircraft is to test whether an eight-wing flapping aircraft can hover in confined spaces, descend into forest gaps, and operate in wind that would ground a conventional drone — all capabilities needed for Search and Rescue missions in difficult terrain.

SECTION 2 — HOW THE EIGHT WINGS WORK TOGETHER

The wings do not all flap at the same time. They flap in a staggered sequence called a metachronal cascade. W1 goes first. Then W2 starts its downstroke a moment later. Then W3, then W4. This staggering means each rearward pair can capture energy from the wake (disturbed air) left by the pair ahead, the same way pelicans in formation save energy by flying in each others wake.

Research on plesiosaurs — ancient marine reptiles with four flippers — found that correctly phased hind flippers generated 60% more thrust and 40% better efficiency than when working alone. The MCO-1D uses the same principle with four pairs instead of two.

Each wing has five independently controlled parameters:

Parameter	What It Does
Amplitude	How far the wing swings in its arc. 0–96°. Hard limit 96° (4° from the mechanical stop).
Frequency	How many times per second the wing flaps. 0.5–3.0 Hz. Hover is estimated near 2.5 Hz. Hard limit 3.0 Hz.
Angle of Attack (AoA)	The pitch of the wing surface relative to airflow. 0–22°. The S1223 wing profile reaches maximum lift at 18°. Above 22° the flow separates and lift drops sharply.
Sweep	The fore-aft position of the wing. Usually fixed. Moves dynamically in the figure-eight stroke mode (H-16).
Dihedral	The up-down tilt of the wing. Used in steep approach mode (pigeon-wing braking). Open-loop.

SECTION 3 — THE SENSING SYSTEM: WHAT THE COMPUTER KNOWS

Most aircraft know their attitude (roll, pitch, heading) and airspeed. The MCO-1D knows those things too, but it also knows the actual aerodynamic state of each individual wing, in real time. This is unprecedented in any flapping-wing aircraft.

What is Known	How It is Measured
Angle of Attack per wing	The MCP3208 ADC chip #1 reads a voltage from a potentiometer inside each pitch servo that indicates the servos actual position. This tells the computer the actual AoA of each wing to 0.044° resolution, updated 50 times per second.
Sweep position per wing	The MCP3208 ADC chip #2 does the same for the sweep servos. The computer knows the actual fore-aft position of each wing.
Stroke position per wing	The ODrive motor controllers track the angle of each motor shaft. Dividing by the 7:1 gear ratio gives the actual sector wheel angle — where the wing is in its flapping arc. No new hardware needed: the ODrive already knows this and reports it over the CAN bus.
Effective Angle of Attack	Computed from geometric AoA + the velocity the wing tip is moving through the air due to flapping. At mid-stroke, the tip can move at 6+ m/s, which adds significantly to the AoA the aerodynamics sees.
Force per wing	Estimated from effective AoA, stroke velocity, and airspeed. The computer knows approximately how much lift and thrust each wing is generating at every moment.

SECTION 4 — HOW THE SOFTWARE CONTROLS THE AIRCRAFT

The main software is called the Wave Library — 45,229 lines of Python that run on the Raspberry Pi. Every 20 milliseconds (50 times per second) it does the following:

Order	What Happens	Details
1	Read sensors	IMU, barometer, rangefinder, OAT, RC receiver, MAVLink from Pixhawk
2	Read per-wing state	MCP3208 ADCs for AoA and sweep, ODrive CAN for stroke position and current
3	Run 58 software layers	Each layer adds one capability — stall detection, gust response, optimizer, active damping, descent control, and so on
4	Compute wing commands	What amplitude, frequency, AoA, and sweep each of 8 wings should do
5	Send commands	8 servo PWM signals (pitch), 8 servo PWM signals (sweep), 8 ODrive velocity commands via CAN
6	Log and report	Write to CSV log file, send telemetry to ground station laptop

The optimizer continuously tries small variations on the current wing configuration, measures the result, and keeps improvements. Over many flights it builds a map of the best configuration for every condition. This map is saved to the SD card and persists between flights.

SECTION 5 — SAFETY RULES

■ WARNING: The frequency ceiling of 3.0 Hz cannot be overridden. The amplitude ceiling of 96° cannot be overridden. The AoA stall protection cannot be overridden. These are hard limits written into the software.

■ WARNING: NEVER attempt TC-02 (first flight) unless TC-01 (bench test) has fully passed. Never attempt TC-03 unless TC-02 is complete. Never add two unknowns at the same time.

✓ The warning systems inform. The optimizer suggests. The pilot decides everything. No mode removes the pilot from the loop.

PART II — HARDWARE: EVERY COMPONENT EXPLAINED

SECTION 6 — WING ASSEMBLIES

What a wing assembly is: one motor → one rope → one sector wheel → one wing. Four assemblies like this per pair means four per pair, eight total across the aircraft. Port and starboard on the same pair are completely independent. They do not share any mechanical part.

Motor model	GoolRC 5010-14, 360 KV, 7 pole pairs (12N14P)
What KV means	360 revolutions per minute per volt at no load
Pole pairs	7 — this number must be entered into the ODrive exactly
Gear ratio	7:1 — the sector wheel turns once per 7 motor turns
Sector wheel arc	100° mechanical limit. Software stops at 96°
Flapping amplitude	The arc the wing moves through. Max 96°. Zero amplitude = wing stopped.
Rope drive	Length of cord connecting motor drum to sector wheel. Acts as a mechanical fuse: slips before anything breaks. Inspect for fraying before every flight.

SECTION 7 — ODRIVE MOTOR CONTROLLERS

An ODrive is a motor controller. It takes a velocity command from the computer and drives a motor to spin at that speed. Each ODrive v3.6 board has two independent motor outputs: axis0 and axis1. The MCO-1D uses four ODrive boards, one per wing pair.

This is the assignment — you must wire the motors exactly this way:

Board	CAN Node	Axis 0	Wing	Axis 1	Wing
Board 0	CAN node 0	axis0	W1 port motor	axis1	W1 starboard motor
Board 1	CAN node 1	axis0	W2 port motor	axis1	W2 starboard motor
Board 2	CAN node 2	axis0	W3 port motor	axis1	W3 starboard motor
Board 3	CAN node 3	axis0	W4 port motor	axis1	W4 starboard motor

The boards connect to the Raspberry Pi via a CAN bus — a single pair of wires (CAN-H and CAN-L) that all four boards share. The Pi talks to all four boards simultaneously on this bus. Each board has a unique node ID (0, 1, 2, 3) so the Pi can address them individually.

● CAUTION: The vel_limit parameter must be set to 150 rad/s on BOTH axis0 AND axis1 on ALL FOUR boards (8 axes total). The factory default is lower and will prevent the wings from reaching operating frequency. This is set by running mco_odrive_setup.py.

SECTION 8 — PITCH SERVOS AND ANGLE OF ATTACK

Each wing has one pitch servo. The pitch servo rotates the wing about its spanwise axis, changing the angle of attack (AoA) — the angle between the wing surface and the oncoming air. Higher AoA = more lift, up to the stall limit of 18-22°.

The servo must have a feedback potentiometer output pin. This is a small voltage (typically 0.5–2.5 V over the servo travel) that tells the computer the servos actual position. This is not the same as the command signal. It is a separate output wire.

GPIO PWM pins (Pi output)	W1p=18, W1s=23, W2p=24, W2s=25, W3p=12, W3s=16, W4p=20, W4s=21
PWM frequency	333 Hz — high-speed digital mode
Supply voltage	7.4 V from dedicated servo LiPo
Feedback pin connection	Servo feedback wire → 1 kΩ resistor → MCP3208 #1 ADC input
Channel assignment	W1p→CH0, W1s→CH1, W2p→CH2, W2s→CH3, W3p→CH4, W3s→CH5, W4p→CH6, W4s→CH7

SECTION 9 — SWEEP SERVOS AND FORE-AFT POSITION

Each wing also has one sweep servo. The sweep servo moves the wing fore and aft along the fuselage axis. In normal flight, the sweep servo holds a fixed position. In two specific experimental modes (H-16 figure-eight and H-18 sweep-clap) it moves dynamically during each stroke.

GPIO PWM pins (Pi output)	W1p=4, W1s=17, W2p=27, W2s=22, W3p=5, W3s=6, W4p=13, W4s=19
PWM frequency	50 Hz — standard
Supply voltage	7.4 V from dedicated servo LiPo
Feedback pin connection	Servo feedback wire → 1 kΩ resistor → MCP3208 #2 ADC input
Channel assignment	W1p→CH0, W1s→CH1, W2p→CH2, W2s→CH3, W3p→CH4, W3s→CH5, W4p→CH6, W4s→CH7

SECTION 10 — THE ADC CHIPS: PER-WING FEEDBACK SENSING

ADC stands for Analogue-to-Digital Converter. The MCP3208 is an ADC chip made by Microchip. It reads a voltage and converts it to a number the computer can use. The MCO-1D uses two of these chips — one to read all eight pitch servo positions, one to read all eight sweep servo positions.

Both chips connect to the same SPI Bus 1 on the Raspberry Pi. SPI (Serial Peripheral Interface) is a way for the Pi to talk to multiple devices over a shared set of wires, telling each device apart by a separate chip select wire. Chip #1 (pitch) uses CE1 (chip enable 1). Chip #2 (sweep) uses CE0.

Both chips share	SPI1: CLK=GPIO11, MOSI=GPIO10, MISO=GPIO9
Pitch ADC (MCP3208 #1)	CE1 = GPIO7 (SPI1 CE1)
Sweep ADC (MCP3208 #2)	CE0 = GPIO8 (SPI1 CE0)
Power	VDD and VREF both connected to Pi 3.3 V
Ground	GND to Pi GND

Resolution	12-bit = 4096 steps over 0–3.3 V = 0.8 mV per step
Config.txt line	dtoverlay=spi1-2cs (enables SPI1 with CE0 and CE1)
Series resistors	1 kΩ on each of the 16 signal lines for protection

SECTION 11 — FLIGHT COMPUTER: RASPBERRY PI 4

The Raspberry Pi 4 is a credit-card-sized computer. It runs the Wave Library Python program at 50 Hz. It also runs background threads for the ground station server, the data logger, and the MAVLink sensor bridge. The 4 GB RAM version is required because the combined memory use of all these systems during a long Discovery Mode flight exceeds 500 MB.

Model required	Raspberry Pi 4 Model B, 4 GB RAM
OS	Raspberry Pi OS Lite 64-bit (no desktop — saves memory and CPU)
Storage	64 GB microSD, Class 10 or A1 rated high-endurance
Control loop	50 Hz — one tick every 20 ms
Mean tick time (Pi 4)	~13 ms — 1.5× headroom before budget overrun
Auto-start	mco-flight.service starts the Wave Library on boot
SSH access	Required for ground station connection and configuration

SECTION 12 — SENSORS

MPU-6050 IMU	I2C addr 0x68. Roll, pitch, yaw rates and accelerations at 100 Hz. Primary attitude reference. Used by stall detection and gust response.
BMP280 barometer	I2C addr 0x76. Altitude above launch point. Active above 3 m where lidar loses terrain accuracy.
TF-Luna lidar	50 Hz. Height 0.1–8 m above ground or water. Primary height reference for landing. Over water: reads wave surface.
DS18B20 OAT	GPIO4, One-Wire. Outside air temperature. Density altitude correction. Need 4.7 kΩ pull-up to 3.3 V.
HX711 + load cell	GPIO26 (DT), GPIO7 (SCK). Static thrust measurement. Bench fixture only — not flown. Requires mco_loadcell.py.

SECTION 13 — POWER SYSTEM

■ WARNING: Use a dedicated 5V BEC for the Raspberry Pi. Motor PWM switching noise on a shared Pi power rail causes SD card write errors and corrupts the flight log and knowledge registry.

Main battery	4S LiPo, 4,000 mAh+. Powers 8 motors via 4 ODrives.
Servo battery	2S LiPo, 1,500 mAh. Dedicated 7.4 V for all 16 servos.
Pi supply	BEC 5 V / 3 A. Isolated from motor supply.
Main fuse	30 A blade fuse on main battery positive lead.
Pi fuse	5 A blade fuse on BEC output.
Low battery warning	30% → amber on panel. 20% → red, land now. Discovery Mode exits automatically at 20%.

SECTION 14 — RADIO SYSTEM

RC receiver	SBUS or IBUS output. 4 channels minimum. CH1=Roll, CH2=Pitch, CH3=Lever, CH4=Yaw. SBUS to Pi UART (GPIO14/15). Bind before first power-on.
Pixhawk	Used as SENSOR BRIDGE ONLY. Does not run flight control. Provides GPS, compass, extra IMU via MAVLink to Pi UART at 921,600 baud. Connect Pixhawk TELEM2 to Pi UART.
Lever (CH3) meaning	Forward = power. Always. This never changes regardless of mode.
Lever 88-100%	Sprint/takeoff — demanding input, maximum performance only
Lever 50%	Hover/neutral in flight
Lever 12-50%	Approach and airbrake zone
Lever 0-12%	Steep approach — pigeon-wing dihedral activates

SECTION 15 — PANEL AND GROUND STATION

Touchscreen panel	Runs mco_panel.py on the Pi. Shows: PRE-FLIGHT, ARM, DISCOVERY, POST-FLIGHT buttons. Plus 6 margin bars (amber=50%, red=25%).
Ground station laptop	Runs mco_ground_station.py. Connects to Pi via WiFi or ethernet on port 5760. Shows: live telemetry, per-wing sensing panel, optimizer status, command prompt.
Command prompt	Type any of 114 GS commands. Examples: arm, disarm, disc_mode_engage, stall_detector_status, lift_balance_report, servo_health_report, stroke_position_status.
Telemetry rate	10 Hz from aircraft to ground station

SECTION 16 — WIRING: CAN BUS, SPI BUS, GPIO

16.1 CAN Bus

```
Pi SPI0 CE0 → MCP2515/MCP2518FD adapter → [CAN bus]
CAN bus: ODrive 0 → ODrive 1 → ODrive 2 → ODrive 3 → 120Ω
At Pi adapter end: also 120Ω between CAN-H and CAN-L
```

The CAN bus is two wires: CAN-H and CAN-L. They must be twisted together. All four ODrive boards connect in a chain. Both ends of the chain need a 120 Ω termination resistor between CAN-H and CAN-L. Without these resistors the signals reflect and the bus will not work.

16.2 SPI Bus 1 (both MCP3208 ADCs)

```
GPIO11 = SPI1 CLK (shared by both ADC chips)
GPIO10 = SPI1 MOSI (shared by both ADC chips)
GPIO9 = SPI1 MISO (shared by both ADC chips)
GPIO7 = SPI1 CE1 (selects pitch ADC, MCP3208 #1)
GPIO8 = SPI1 CE0 (selects sweep ADC, MCP3208 #2)
```

16.3 Feedback Signal Wires

Each servo feedback wire goes through a 1 kΩ series resistor before connecting to the ADC input. Solder the resistor in-line on the wire, near the ADC chip, not near the servo. This prevents damage if the wire accidentally shorts to the 5 V rail.

16.4 GPIO Reference Table

Pin / Bus	Function
GPIO 4	DS18B20 OAT sensor (One-Wire)
GPIO 7	SPI1 CE1 — MCP3208 #1 pitch ADC chip select
GPIO 8	SPI1 CE0 — MCP3208 #2 sweep ADC chip select
GPIO 9	SPI1 MISO — both ADC chips
GPIO 10	SPI1 MOSI — both ADC chips
GPIO 11	SPI1 CLK — both ADC chips
GPIO 12	W3p pitch servo PWM output
GPIO 13	W4p sweep servo PWM output
GPIO 16	W3s pitch servo PWM output
GPIO 17	W1s sweep servo PWM output
GPIO 18	W1p pitch servo PWM output
GPIO 19	W4s sweep servo PWM output
GPIO 20	W4p pitch servo PWM output
GPIO 21	W4s pitch servo PWM output
GPIO 22	W2s sweep servo PWM output
GPIO 23	W1s pitch servo PWM output
GPIO 24	W2p pitch servo PWM output

GPIO 25	W2s pitch servo PWM output / CAN adapter interrupt
GPIO 26	HX711 load cell DT (data) — bench use only
GPIO 27	W2p sweep servo PWM output
SPI0 CE0	MCP2515/2518 CAN adapter
I2C1 (GPIO2/3)	IMU (0x68) and barometer (0x76)
UART (GPIO14/15)	RC receiver SBUS + Pixhawk MAVLink 921,600 baud

Step	Action	Expected Result / Notes
1	Mount airframe	All structural members assembled. Verify rigidity. No flex between mounting points for sector wheel axles.
2	Mount 8 sector wheels	One per wing. Rotate by hand through full arc. Must be smooth, no binding, no roughness.
3	Mount 8 motors	One per wing. Shaft aligned with rope drum. Apply thread lock to all motor mount screws.
4	Install 8 rope drives	One per wing. Verify 7:1 gear ratio by counting: turn motor shaft 7 times, verify sector turns exactly once.
5	Mount 8 wings	Attach to sector wheel pivot points. Verify each wing swings through full 96° without binding.
6	Set pitch servo neutral positions	Before attaching servo horns: apply power to servo rail only (no motors). Command all pitch servos to 0°. Verify servo is at mechanical neutral. Attach horn.
7	Install 8 pitch servos	One per wing. Set neutral before mounting. Thread feedback wire back along servo cable.
8	Set sweep servo neutral positions	Same procedure as step 6 for sweeps. Command to 60° (midpoint). Attach horn.
9	Install 8 sweep servos	One per wing. Set neutral before mounting.
10	Mount 4 ODrive boards	Board 0 near W1, Board 3 near W4. Secure with vibration isolation. Connect motor phases: Board N axis0 = W(N+1) port, Board N axis1 = W(N+1) stbd.
11	Wire CAN bus	Daisy-chain: Pi CAN adapter → Board 0 → 1 → 2 → 3. Use twisted pair CAN-H and CAN-L. 120 Ω terminator at Pi end AND at Board 3 end.
12	Wire pitch servo PWM	Pi GPIO to pitch servo signal wires per Section 16.
13	Wire sweep servo PWM	Pi GPIO to sweep servo signal wires per Section 16.
14	Wire pitch servo feedback	Each pitch servo feedback pin → 1 kΩ resistor → MCP3208 #1 (SPI1 CE1) channel per Section 16.
15	Wire sweep servo feedback	Each sweep servo feedback pin → 1 kΩ resistor → MCP3208 #2 (SPI1 CE0) channel per Section 16.
16	Mount Pi 4 and sensors	Secure Pi to airframe with vibration isolation. Connect IMU and barometer to I2C1 (GPIO2 SDA, GPIO3 SCL). Mount lidar rangefinder facing downward, clear of wing arc.
17	Connect OAT sensor	DS18B20 data → GPIO4. 4.7 kΩ pull-up from data wire to Pi 3.3 V.
18	Wire power system	Main battery → 30 A fuse → ODrives. Servo battery → all servo rails. BEC → Pi. DO NOT connect batteries yet — just route cables.
19	Install RC receiver	SBUS output → Pi UART (GPIO15 RXD). Bind receiver to transmitter now (follow transmitter manual).

20	Connect Pixhawk	Pixhawk TELEM2 → Pi UART at 921,600 baud.
21	First power-on — Pi only	Connect 5 V BEC to Pi only, no motors, no servos. Verify Pi boots. SSH in. Run: python3 --version (should show 3.8+).
22	Install software	See Part IV. Do not proceed to Step 23 until Part IV is complete.
23	Run ODrive setup	python3 mco_odrive_setup.py Configures all 4 boards, all 8 axes. Takes ~2 minutes.
24	Calibrate ADC feedback	For each of 16 servos: command to 3 positions, read ADC voltage, run calibrate_servo_feedback(). See TC-01-14 to TC-01-16.

PART III — ASSEMBLY SEQUENCE

SECTION 17 — ASSEMBLY STEPS 1 THROUGH 24

■ WARNING: Do not connect any battery until Step 18. Do not power the ODrives until mco_odrive_setup.py has been run.

Step	Action	Expected Result / Notes
1a	Set hostname	Type: mco-1d
1b	Enable SSH	Tick the box. This lets your laptop connect to the Pi.
1c	Set username and password	Username: pi Password: choose something strong
1d	Configure WiFi	Your field network SSID and password, OR leave blank and use ethernet
1e	Set locale	Your timezone and keyboard layout

Step	Action	Expected Result / Notes
6a	Connect to all 4 ODrives	Contacts each board on CAN node IDs 0, 1, 2, 3
6b	Configure axis0 on each board	pole_pairs=7, MOTOR_TYPE_HIGH_CURRENT, vel_limit=150 rad/s, velocity control mode
6c	Configure axis1 on each board	Same settings as axis0
6d	Save to non-volatile memory	Settings persist after power cycle
6e	Verify	Reads back settings and confirms they were saved

Step	Action	Expected Result / Notes
V-01	Wave Library imports	$ python3 -c "import mco_wave_library; print(OK)" → must print OK
V-02	CAN bus	$ ip link show can0 → must show UP
V-03	ODrives	$ python3 mco_odrive_setup.py --verify → all 4 boards, 8 axes
V-04	IMU	$ python3 -c "import smbus; b=smbus.SMBus(1); print(b.read_byte(0x68))" → number appears
V-05	Pitch ADC	$ python3 -c "import mco_wave_library as w; print(w.read_servo_feedback_adc())" → 8 floats, not all None
V-06	Sweep ADC	Same as V-05 but check MCP3208 #2 is also responsive
V-07	OAT sensor	$ cat /sys/bus/w1/devices/28-*/w1_slave → temperature reading at end
V-08	SIM mode	$ python3 mco_main.py --sim → no errors, "flight loop starting" message
V-09	Ground station	$ python3 mco_ground_station.py → connects, shows telemetry
V-10	All 16 servos	$ python3 mco_odrive_setup.py --servo-test → all 16 servos deflect and return

V-11	Log file	After 10 seconds of sim mode: ls /home/pi/logs/ → CSV file created

PART IV — SOFTWARE INSTALLATION

The software installation has eight steps. Do them in order. Every command shown here must be typed exactly as written. The dollar sign ($) means this is a command you type into a terminal. Do not type the dollar sign itself.

STEP 1 — INSTALL RASPBERRY PI OS

On your laptop, download the Raspberry Pi Imager from raspberrypi.com/software. Insert the 64 GB microSD card into your laptop. Open the Imager and choose:

Setting	Value
Device	Raspberry Pi 4
OS	Raspberry Pi OS Lite (64-bit) — choose the Lite version, not the full desktop version
Storage	Your 64 GB microSD card

Click the gear icon (Advanced Options) and configure:

Click Write. Wait for it to finish (takes 5-10 minutes). Insert the SD card into the Pi.

STEP 2 — CONFIGURE /boot/config.txt

Before booting the Pi, the SD card needs three additions to the configuration file. Open the SD card on your laptop (it will appear as a drive called "boot") and open the file config.txt in a text editor. Add these three lines at the very end of the file:

```
# CAN bus adapter (MCP2515 or MCP2518FD)
dtoverlay=mcp2515-can0,oscillator=8000000,interrupt=25
# Both MCP3208 ADC chips on SPI Bus 1 (pitch ADC on CE1, sweep on CE0)
dtoverlay=spi1-2cs
# Outside air temperature sensor (DS18B20 on GPIO4)
dtoverlay=w1-gpio,gpiopin=4
```

Save the file. Also add these lines to enable I2C and SPI at boot:

```
dtparam=i2c_arm=on
dtparam=spi=on
```

✓ If you have an MCP2518FD CAN adapter (instead of MCP2515), the overlay name is different. Check your adapter documentation.

STEP 3 — INSTALL PYTHON PACKAGES

Boot the Pi. SSH into it from your laptop:

```
$ ssh pi@mco-1d.local
```

If that does not work, find the Pis IP address on your router and use:

```
$ ssh pi@192.168.x.x (replace x.x with actual IP)
```

Once logged in, run:

```
$ pip3 install pymavlink python-can pigpio spidev --break-system-packages
$ sudo systemctl enable pigpiod && sudo systemctl start pigpiod
```

pigpiod is a background service that controls the Pi GPIO pins for servo PWM. It must be running for the servos to work.

✓ The --break-system-packages flag is required on Raspberry Pi OS Bookworm and later. It allows pip to install into the system Python.

STEP 4 — COPY SOFTWARE FILES

Copy all software files from your laptop to the Pi. From your laptop (not from inside the Pi SSH session):

```
$ scp mco_wave_library.py mco_main.py mco_odrive_interface.py \
mco_servo_interface.py mco_mavlink.py mco_gs_server.py \
mco_logger.py mco_odrive_setup.py mco_preflight.py \
mco_panel.py mco_hypothesis_tool.py mco_sync.py \
mco_can_setup.sh mco_install.sh \
pi@mco-1d.local:/home/pi/
```

Then SSH back in and create the required directories:

```
$ ssh pi@mco-1d.local
$ mkdir -p /home/pi/logs /home/pi/knowledge
$ chmod +x /home/pi/mco_can_setup.sh /home/pi/mco_install.sh
```

Run the install script:

```
$ bash /home/pi/mco_install.sh
```

This creates the systemd service files and enables auto-start on boot.

STEP 5 — CONFIGURE THE CAN BUS

Reboot the Pi first (the config.txt changes from Step 2 take effect on reboot):

```
$ sudo reboot
```

After reboot, SSH back in and run the CAN setup script:

```
$ sudo bash /home/pi/mco_can_setup.sh
```

You should see: "CAN-FD interface can0: UP at 1Mbit/s". If not, check your CAN adapter wiring and the dtoverlay line in config.txt.

Verify the interface is up:

```
$ ip link show can0
```

It should show "UP" in the output. If it shows "DOWN", run the setup script again.

STEP 6 — CONFIGURE ODRIVES

The ODrives need to be configured before the motors will respond correctly. Connect the ODrives to the CAN bus. Apply main battery power. Then run:

```
$ python3 /home/pi/mco_odrive_setup.py
```

This script will:

■ WARNING: If any board does not respond, check CAN bus wiring. Check that both 120 Ω termination resistors are installed. Run: candump can0 to see if any CAN traffic is visible.

STEP 7 — VERIFY EVERYTHING WORKS

Run each of these checks in order. All must pass before proceeding to TC-01.

STEP 8 — SET UP THE GROUND STATION LAPTOP

On your laptop (not the Pi):

```
$ python3 mco_ground_station.py
```

or with explicit connection details:

```
$ python3 mco_ground_station.py mco-1d.local 5760
```

You should see a connection message and then live telemetry updating at 10 Hz. The command prompt appears at the bottom. Type any command and press Enter. Type "help" to see all 114 available commands.

To run the sensing display panel (shows per-wing AoA, force waveform, etc.):

```
>>> sensing_display
```

Step	Action	Expected Result / Notes
TC-01-01	Servo neutral check	Command all 16 servos to their defined neutral positions. Measure actual angle with a protractor. Record any deviation. Adjust trim offsets in the library if deviation exceeds ±2°. PASS: all 16 servos within ±2° of commanded neutral.
TC-01-02	Servo range check	Command each servo to +45° then -45°. Verify smooth travel without binding. PASS: all 16 servos reach full commanded range.
TC-01-03	Pitch ADC verification	Run: python3 -c "import mco_wave_library as w; print(w.read_servo_feedback_adc())" While moving each pitch servo manually, verify the reading changes. PASS: all 8 channels return non-None values that change with servo position.
TC-01-04	Sweep ADC verification	Same as TC-01-03 but for sweep servos and MCP3208 #2. PASS: all 8 sweep channels return changing values.
TC-01-05	ODrive 8-axis communication	Run: python3 mco_odrive_setup.py --verify PASS: all 4 boards respond, all 8 axes show correct settings.
TC-01-06	Motor spin test — low speed	Command each of 8 axes to 10 rad/s. Listen for bearing noise. Check for unusual vibration. PASS: all 8 motors spin smoothly without noise or vibration.
TC-01-07	Wing flapping at 0.5 Hz	Command all 8 wings to 0.5 Hz. Run 30 seconds. Inspect all 8 rope drives for slip, fraying, or wear. PASS: smooth mechanical motion, no rope problems.
TC-01-08	Wing flapping at operating frequency	Command all 8 wings to 2.5 Hz (estimated hover frequency). Run for 60 seconds. Monitor motor temperatures with IR thermometer. Check all rope drives again. PASS: sustained 2.5 Hz, all motor temperatures below 60°C.
TC-01-09	Pitch ADC calibration — all 8 pitch servos	For each pitch servo: command to -45°, read ADC voltage and record. Command to 0°, read and record. Command to +45°, read and record. Then call: calibrate_servo_feedback(wing_idx, v_at_minus45, v_at_0, v_at_plus45) Repeat for all 8 wings (indices 0-7). PASS: calibration accepted for all 8, round-trip error < 1°.
TC-01-10	Sweep ADC calibration — all 8 sweep servos	Same procedure as TC-01-09 for sweep servos. PASS: calibration accepted for all 8 sweep servos.
TC-01-11	Frequency ceiling test	Step frequency from 2.0 to 3.0 Hz in 0.1 Hz steps. Hold each step for 30 seconds. Record motor temperature at each. Identify the maximum sustained frequency before thermal limiting. PASS: aircraft can sustain at least 2.5 Hz for 60 seconds continuously.

TC-01-12	Static thrust measurement	Secure aircraft to load cell test fixture. Command 2.5 Hz and record thrust. Vary from 1.5 to 3.0 Hz in 0.25 Hz steps. Record thrust at each frequency. This produces the lift-frequency curve. PASS: thrust exceeds 33.8 N (MTOW weight) at some frequency ≤3.0 Hz.
TC-01-13	RC channel mapping verification	Move each stick to its extreme position. Verify on ground station: CH1=Roll, CH2=Pitch, CH3=Lever, CH4=Yaw. PASS: all channels correctly assigned.
TC-01-14	Safety system verification	Command frequency >3.0 Hz — verify hard cap activates at 3.0 Hz. Command amplitude >96° — verify hard cap activates at 96°. Disconnect one motor lead — verify fault detection fires within 0.2 seconds. PASS: all three safety systems confirmed active.
TC-01-15	Symmetry check	Command all 8 wings to the same AoA (15°). Run: lift_balance_report on ground station. PASS: port and starboard estimated lift within 5% of each other.
TC-01-16	Fill Build Measurement Sheet and export knowledge	Record in the measurement sheet: actual hover frequency estimate, frequency ceiling, motor temperatures, servo calibration voltages, rope drive condition notes. Run: knowledge_export_usb to save calibration data. PASS: sheet complete, knowledge exported.

Step	Action	Expected Result / Notes
TC-02-01	First hover attempt	Advance lever slowly from 50% toward 70%. Hold each position 3 seconds. Record the lever position and frequency when lift-off occurs. Land. Repeat 3 times to confirm repeatability. Record: hover lever position, hover frequency from telemetry.
TC-02-02	Hover stability assessment	Achieve hover at measured lever position. Hold 30 seconds without correction. Ground station shows sigma_h (altitude deviation). If sigma_h exceeds 0.5 m: investigate before continuing.
TC-02-03	Controlled altitude hold	Hover for 60 seconds. Pilot makes gentle corrections. Ground station records actual hover frequency from freq_hz telemetry. THIS IS THE KEY MEASUREMENT: record hover_freq_hz accurately. Also note what the actual_aoa_deg values are during stable hover.
TC-02-04	Lift-frequency curve in air	From stable hover, vary frequency ±0.3 Hz from hover frequency. Record altitude change rate at each frequency step. This gives dL/df (lift change per Hz change). Record: lift_freq_slope_N_per_Hz for descent controller calibration.
TC-02-05	Roll response test	Apply 20%, 40%, 60% roll stick from stable hover. Record roll rate achieved. Release stick. Record time to settle (rate falls below 2°/s). This data calibrates: ACTIVE_DAMP_K_AMP and ACTIVE_DAMP_SETTLE_TIME_S.

TC-02-06	Calibrate descent controller	With measured values, run on ground station: calibrate_descent_controller (it prompts for hover_freq_hz and slope). Then test: descent_ctrl_engage 100 (100 fpm target). Verify 100 fpm descent is stable and repeatable.
TC-02-07	H-26 sidestep observation (critical unknown)	From stable hover, apply small differential AoA port vs stbd (H-26 mode). OBSERVE: does the aircraft translate sideways while staying level, or does it roll? This is the most uncertain prediction in the programme. Record observation carefully — it determines how H-26 is used.
TC-02-08	First discovery session	Enable auto-discovery from ground station: disc_mode_engage Hold hover for 5 minutes. Allow optimizer to begin building hover map. Run: knowledge_export_usb at end of session. Check servo_health_report and stall_detector_status.

Step	Action	Expected Result / Notes
TC-03-01	Hover-to-forward transition	From stable hover, advance lever toward 70%. Allow pitch forward to develop naturally. Record transition speed and lever position. Return to hover. Repeat 3×.
TC-03-02	Forward flight altitude hold	Achieve forward flight at cruise lever (~65%). Hold altitude using descent controller. Record airspeed from GPS, power consumption from bus voltage.
TC-03-03	Speed envelope sweep	Fly at 60%, 65%, 70%, 75%, 80% lever. Record airspeed and power at each. Identify minimum stable speed (stall speed in forward flight).
TC-03-04	IFR descent validation	descent_ctrl_engage 100 in forward flight. Verify 100 fpm maintained. Test 300 fpm. Validates the standard IFR descent capability.
TC-03-05	First Discovery Mode flight	Fly racetrack pattern at 100 ft AGL. disc_mode_engage for 30 minutes. Export knowledge: knowledge_export_usb
TC-03-06	CT-J curve measurement	Measure thrust coefficient versus advance ratio across speed range. Required for post-TC-03 stroke plane controller development.

PART V — COMMISSIONING TESTS

There are three commissioning test cards. They must be completed in order. TC-01 is entirely on the ground. TC-02 is first flight. TC-03 is forward flight. You cannot skip ahead. Each test card produces data that the next card needs.

■ WARNING: If any TC-01 step fails, identify and fix the problem before continuing. Do not attempt TC-02 until every TC-01 step has passed.

SECTION 26 — TC-01: BENCH TEST (16 STEPS)

TC-01 is a ground test. The aircraft does not leave the bench. All 16 steps must pass before any flight attempt. Two people required: one at the aircraft, one at the ground station laptop.

SECTION 27 — TC-02: FIRST HOVER (8 STEPS)

TC-02 is the first flight. The goal is controlled, stable hover — not performance. Take it slowly. Every advanced system in the Wave Library needs data that only TC-02 can provide.

Site requirements: 50 m × 50 m clear area. No people within 30 m. Wind below 5 knots. No gusts. Sea level to 300 m elevation. Two people minimum: pilot on transmitter, operator at ground station.

TC-02 Critical Calibration Outputs — record these for Wave Library update:

Hover frequency Hz	The actual frequency at which the aircraft holds altitude at MTOW
Lift-freq slope N/Hz	How much lift changes per Hz change near hover
Roll rate °/s per %	Roll rate produced by each % of stick deflection
Settle time seconds	Time from stick release to roll rate below 2°/s
H-26 result	Does differential AoA produce lateral translation or roll?
AoA at hover °	What actual_aoa_deg values appear during stable hover

SECTION 28 — TC-03: FORWARD FLIGHT (6 STEPS)

TC-03 begins only after TC-02 is complete and all calibrations are applied. The pilot must be comfortable with hover control before attempting TC-03.

Step	Action	Expected Result / Notes
1	Area clear	No people within 30 m. Landing zone identified. No overhead obstacles.
2	Weather	Below 15 kt normal. Below 5 kt for TC-02. Check gust forecast.
3	Batteries	Main >90%. Servo >90%. Check voltages on ground station.
4	SD space	ls /home/pi/logs/ — verify at least 500 MB free.
5	ODrives	Ground station shows all 4 boards and 8 axes IDLE.
6	ADC check	servo_health_report — all 8 wings show Healthy.
7	RC signal	Ground station shows RC VALID with correct channel mapping.
8	IMU	Place aircraft level. IMU reads 0° pitch, 0° roll.
9	Knowledge	discovery_status — knowledge registry loaded and current.
10	Margins	All 6 margin bars green on panel.
11	ARM	Press ARM button on panel. Verify armed state. Make small stick inputs — verify correct response directions.

Step	Action	Expected Result / Notes
1	DISARM	Press DISARM. All motors stop.
2	POST-FLIGHT	Press POST-FLIGHT button. 8-minute motor cooling period.
3	Knowledge export	knowledge_export_usb — saves discovery registry.
4	Log retrieval	scp pi@mco-1d.local:/home/pi/logs/ ./ from laptop.
5	Battery check	Inspect both batteries for puffing or unusual heat.
6	Rope drives	Inspect all 8 rope drives. Any fraying = replace before next flight.
7	Servo health	servo_health_report — look for any wings with rising tracking RMS.
8	Airframe	Visual inspection for cracks, loose fasteners, worn linkage.

PART VI — FLIGHT OPERATIONS

SECTION 29 — PRE-FLIGHT CHECKLIST

SECTION 30 — NORMAL FLIGHT

Takeoff	Advance lever smoothly from 50% toward hover lever position found in TC-02. Do not jerk — smooth input lets the optimizer adapt. Aircraft will lift off. Hold hover position.

Hover	Hold lever at hover position. Small roll and yaw for positioning. The trim balance system ensures port and stbd contribute equally. Watch sigma_h on ground station.
Forward flight	Advance lever toward 65-70% from hover. Allow pitch forward to develop. Hold altitude with lever. Engage descent_ctrl_engage for precise altitude hold.
Landing	Reduce lever toward 50% for descent. For precision: descent_ctrl_engage 100 (100 fpm). Do not cut lever abruptly — aircraft needs a few wingbeats to lose momentum cleanly.

SECTION 31 — EMERGENCY PROCEDURES

■ WARNING: LOSS OF CONTROL: Reduce lever toward 50%. If aircraft continues to behave unexpectedly: full lever aft. Disarm if uncontrollable. A controlled descent to hard landing is better than endangering people.

Wing failure in flight	Read urgency level from ground station. CONTINUE WITH CAUTION (7 wings): land when convenient. LAND AS SOON AS PRACTICABLE (5-6): within 2 minutes. LAND IMMEDIATELY: put it down now, any surface.
Battery low — amber (30%)	Land at first convenient opportunity.
Battery low — red (20%)	Land now. Discovery Mode exits automatically.
RC signal loss	Aircraft holds last config for 2 seconds then descends at 300 fpm. Restore RC signal to resume control.
Rope slip alert	Land as soon as practicable. Inspect all rope drives before next flight. Rope_slip_detected in soft faults.

SECTION 32 — POST-FLIGHT

Step	Action	Expected Result / Notes
1	Achieve stable flight	Hover or forward flight, all margins green.
2	Engage Discovery Mode	disc_mode_engage (uses defaults: 100 ft AGL, 60 min)
3	Monitor	disc_mode_status every few minutes. Watch margin bars.
4	Exit	disc_mode_exit or wait for duration to expire.
5	Export	knowledge_export_usb (runs automatically on exit too).

PART VII — DISCOVERY PROGRAMME

SECTION 33 — WHAT DISCOVERY MODE DOES

The optimizer runs continuously in every flight mode, quietly testing small variations. Discovery Mode is what happens when you give the optimizer the entire flight with no competing demands. The aircraft holds a racetrack pattern and runs four exploration systems at full rate:

Layer 1 — Gradient	Small improvements to the current best configuration. Always running.
Layer 2 — Biomimetic	Tests all 29 hypotheses (109 variants) from biological research.
Layer 3 — Evolutionary	Breeds new configurations from two previously good ones.
Layer 4 — Random	Escapes local optima by jumping to random configurations.

The three-axis trial validity gate ensures only accurately executed configurations are scored. If AoA error exceeds 3°, sweep error exceeds 5°, or stroke phase error exceeds 10°, the trial is discarded rather than corrupting the discovery registry with misleading data.

SECTION 34 — RUNNING A DISCOVERY FLIGHT

Focus modes (for specific experiment types):

Command	Focus
disc_mode_engage_focused("biomimetic")	All 29 hypotheses at maximum rate
disc_mode_engage_focused("hover_endurance")	Minimum power for SAR loiter
disc_mode_engage_focused("precision")	Minimise sigma_h, sigma_roll, sigma_hdg
disc_mode_engage_focused("aerobatic")	Crispness and hesitation hold quality
disc_mode_engage_focused("gust_resilience")	Stability in real turbulence
disc_mode_engage_focused("fault_tolerance")	Wing simulation and progressive chains

SECTION 35 — THE 29 HYPOTHESES

ID	Name	Description
H-01	Dragonfly phase	Phase switching: in-phase for force, anti-phase for stability. 5 variants.
H-02	Bat advance rotation	AoA transition before stroke reversal. Direct Re match. 5v.

H-03	Manta spanwise wave	Phase gradient root-to-tip. 89% prop. efficiency. 3v.
H-04	Pterosaur soar	0.8 Hz, 18° AoA (Re-corrected). Needs h>30m. 1v.
H-05	Hawkmoth LEV	Amplitude gradient delays LEV breakdown. 4v.
H-06	Negative cascade	W4 leads W1 — reversed cascade. 1v.
H-07	Split frequency	10% inter-pair frequency differential. 3v.
H-08	Manta Strouhal	Speed-adaptive St 0.20–0.40 — now measured from actual stroke. 1v.
H-12	Locust reversal burst	ODrive burst at stroke reversal. 3v.
H-13	Plesiosaur phase	Speed-adaptive cascade phase (Muscutt 2017). 4v.
H-14	Plesiosaur brake	Anti-augmentation deliberate drag. 1v.
H-15	Plesiosaur dominance	Pair amplitude dominance switching. 6v.
H-16	Turtle figure-eight	Dynamic sweep per stroke — needs sweep ADC. 4v.
H-17	Turtle asymmetric	Power downstroke, recovery upstroke. 1v.
H-18	Turtle sweep-clap	Sweep-axis clap at downstroke bottom. 3v.
H-19	Sea lion AoA	15° thrust / 22° lift (Re-corrected from sea lion data). 4v.
H-20	Otter advance ratio	Drag-to-lift mode switch with speed. 5v.
H-21	Sea lion fling	High-velocity clap initiation. 3v.
H-22	Bat ventral clap	Upstroke ventral clap — double lift peak. Direct Re. 4v.
H-23	Bat speed-adaptive AoA	-8° hover → +5° cruise → -3° sprint. Direct Re. 6v.
H-24	MCO-1D: AoA gradient	W1 at thrust AoA, W4 at lift AoA. No prior precedent. 4v.
H-25	MCO-1D: Per-stroke roll	AoA differential for roll, 4× faster than amplitude. 4v.
H-26	MCO-1D: Force decouple	Sidestep without banking — unknown until TC-02. 4v.
H-27	MCO-1D: Servo bandwidth	Maps servo tracking speed at each frequency — needs MCP3208. 12v.
H-28	MCO-1D: Asym cascade	Separate port and stbd cascade phases. Yaw or efficiency? 6v.
H-29	MCO-1D: Resonance seek	Each wing finds its own mechanical resonant frequency. 4v.
H-30	MCO-1D: Power smooth	Distribute reversals to minimise bus voltage ripple. 3v.
H-31	MCO-1D: Fourier path	Arbitrary 2D stroke path beyond the figure-eight ellipse. 4v.
H-32	MCO-1D: Multi-wake	W3/W4 catch multiple upstream wakes simultaneously. 2v.

SECTION 36 — WATER OPERATIONS

● CAUTION: Water operations require float hardware not yet specified. Software framework is complete. Hover must be confirmed at TC-02 first. This section describes the planned procedure.

Wave limits	15 in (0.38 m) FAA safe. 18 in (0.46 m) restriction begins.

Departure	Power as floats RISE toward wave crest (FAA H-8083-23 doctrine). NEVER from trough.
Glassy water	descent_ctrl_engage 100. Rangefinder only. Trust instruments.
Porpoising	First oscillation: back pressure. Second: power to idle, full elevator back.

PART VIII — REFERENCE

SECTION 37 — GPIO AND SPI PIN REFERENCE

Already shown in Section 16. Repeated here for field reference.

Pin	Function
GPIO 4	DS18B20 OAT temperature sensor (One-Wire)
GPIO 5	W3p sweep servo PWM
GPIO 6	W3s sweep servo PWM
GPIO 7	SPI1 CE1 — MCP3208 #1 pitch ADC (chip select)
GPIO 8	SPI1 CE0 — MCP3208 #2 sweep ADC (chip select)
GPIO 9	SPI1 MISO — shared by both ADC chips
GPIO 10	SPI1 MOSI — shared by both ADC chips
GPIO 11	SPI1 CLK — shared by both ADC chips
GPIO 12	W3p pitch servo PWM
GPIO 13	W4p sweep servo PWM
GPIO 16	W3s pitch servo PWM
GPIO 17	W1s sweep servo PWM
GPIO 18	W1p pitch servo PWM
GPIO 19	W4s sweep servo PWM
GPIO 20	W4p pitch servo PWM
GPIO 21	W4s pitch servo PWM
GPIO 22	W2s sweep servo PWM
GPIO 23	W1s pitch servo PWM
GPIO 24	W2p pitch servo PWM
GPIO 25	W2s pitch servo PWM / CAN interrupt
GPIO 26	HX711 DT (load cell data — bench only)
GPIO 27	W2p sweep servo PWM
I2C1 GPIO2/3	IMU addr 0x68 and barometer addr 0x76
SPI0 CE0	MCP2515/2518FD CAN bus adapter
UART GPIO14/15	RC receiver SBUS input + Pixhawk MAVLink 921,600 baud

SECTION 38 — ODRIVE AXIS ASSIGNMENT

Board	CAN Node	axis0	axis1
Board 0	CAN node 0	axis0 → W1 port	axis1 → W1 starboard
Board 1	CAN node 1	axis0 → W2 port	axis1 → W2 starboard
Board 2	CAN node 2	axis0 → W3 port	axis1 → W3 starboard
Board 3	CAN node 3	axis0 → W4 port	axis1 → W4 starboard

CAN arbitration ID formula (ODrive v3.6):

```
axis0: arb_id = (node_id << 5) | cmd_id
axis1: arb_id = ((node_id | 0x10) << 5) | cmd_id
Wing index to board+axis: board = wing_idx // 2, axis = wing_idx % 2
```

SECTION 39 — SELECTED GROUND STATION COMMANDS

Command	Function
arm / disarm	Arm or disarm all 8 motor axes
emergency_stop	Immediate stop — all amplitudes to zero
failure_briefing	Pre-flight failure performance table (255 combinations)
fault_summary	Current fault and soft-fault state
servo_health_report	AoA tracking RMS for all 8 wings
stall_detector_status	Per-wing stall likelihood (or per-pair without Tier 3)
lift_balance_report	Port vs stbd actual lift estimate
stroke_position_status	Actual vs commanded stroke for all 8 wings
trim_balance (status)	Port/stbd lift asymmetry percentage
effective_aoa_status	Effective (not geometric) AoA per wing
disc_mode_engage	Start Discovery Mode full cycle
disc_mode_engage_focused X	Focused Discovery Mode (see Section 34)
disc_mode_exit	Exit and auto-export knowledge
disc_mode_status	Current discovery session progress
descent_ctrl_engage N	Engage descent at N fpm (100 = standard IFR)
descent_ctrl_status	Descent controller state
knowledge_export_usb	Copy discovery registry to USB stick
calibrate_descent_controller	Apply TC-02 measurements to descent controller
calibrate_servo_feedback	Per-servo ADC voltage-to-angle calibration
wing_sim_request	Request next wing idle simulation
wing_sim_approve	Approve and arm next simulation test
prog_sim_request	Next progressive failure chain

aerobatic_status	Active damping state and crispness score
precision_status	Current sigma values for h, roll, hdg, sink
force_waveform_status	8-wing instantaneous force data
strouhal_status	Measured Strouhal number per pair

SECTION 40 — SOFTWARE FILE REFERENCE

File	Purpose (lines)
mco_wave_library.py	45,229 lines. Main flight control. 58-layer tick stack. All optimizer, sensing, hypotheses, fault tolerance, aero state engine.
mco_main.py	389 lines. 50 Hz flight loop. Reads all sensors including MCP3208 ADCs and ODrive positions. Applies commands. Logs. Sends telemetry.
mco_odrive_interface.py	253 lines. 4-board 8-axis CAN interface. Handles axis0 and axis1 independently. Reads position and current per wing.
mco_servo_interface.py	110 lines. Pitch and sweep servo PWM via pigpio.
mco_mavlink.py	290 lines. Pixhawk MAVLink background thread. GPS and IMU.
mco_gs_server.py	193 lines. TCP server port 5760. Accepts ground station.
mco_ground_station.py	660 lines. Laptop client. Shows sensing display panel.
mco_logger.py	220 lines. 50 Hz CSV logger. 60+ fields per tick including all per-wing sensing data.
mco_odrive_setup.py	189 lines. Configures all 4 boards, 8 axes.
mco_preflight.py	782 lines. Three-tier pre-flight check. Includes ADC check.
mco_panel.py	485 lines. Touchscreen panel. 4 buttons + 6 margin bars.
mco_hypothesis_tool.py	347 lines. Hypothesis authoring. H-28 to H-32 registry.
mco_sync.py	229 lines. Knowledge transfer between Pis.
mco_can_setup.sh	36 lines. CAN interface shell script.
mco_install.sh	46 lines. One-step installation script.
mco_loadcell.py	PENDING — write before TC-01.

SECTION 41 — CALIBRATION CONSTANTS PENDING TC-02

These constants have estimated values that will work conservatively. After TC-02, replace them with measured values.

Constant	Current Value	How to Calibrate
DESCENT_CTRL_HOVER_FREQ_HZ	2.50 Hz (est)	Measure during TC-02-03. Read freq_hz from telemetry during 60s hover.
DESCENT_CTRL_K_FREQ_PER_FPM	0.0009 (est)	Computed by calibrate_descent_controller() from hover freq and lift slope.
ACTIVE_DAMP_K_AMP	0.40 (est)	Measure during TC-02-05 roll response test. °_amplitude per °/s roll rate.
ACTIVE_DAMP_K_AOA	0.15 (est)	Measure during TC-02-05. AoA correction per °/s roll rate.
ACTIVE_DAMP_SETTLE_TIME_S	0.15 s (est)	Measure during TC-02-05. Time from stick release to rate <2°/s.
AOA_CL_KP	0.30 (est)	AoA closed-loop proportional gain. Tune post-TC-02.

AOA_CL_KI	0.05 (est)	AoA closed-loop integral gain. Tune post-TC-02.
AOA_CL_KD	0.02 (est)	AoA closed-loop derivative gain. Tune post-TC-02.
GR_AOA_FAST_GAIN	0.25 (est)	Gust response AoA fast leg gain. Tune from gust response data at TC-02.
H-26 behaviour	Unknown	Does differential AoA produce lateral translation or roll? TC-02 reveals this.

OUTSTANDING ITEMS BEFORE TC-01

mco_loadcell.py	Write HX711 reader before TC-01. Required for TC-01-12 static thrust measurement.
Build Measurement Sheet	Fill all physical dimensions from assembled hardware. Update SEMI_SPANS_MM, CHORD_MM if measurements differ.
ODrive vel_limit confirmed	Run mco_odrive_setup.py --verify after first power-on to confirm vel_limit=150 was saved on all 8 axes.
RC channel mapping	Verify CH1=Roll, CH2=Pitch, CH3=Lever, CH4=Yaw before arming for the first time.
Float specification	TBD after TC-02 hover confirmation. Water operations cannot be planned until float geometry is known.
Stroke plane controller	Post-TC-03. Requires CT-J curve from TC-03-06.